

TG CPGET 2026 — Machine Learning Revision Handbook

Scope: Introduction · Limits of Learning · Geometry & Nearest Neighbours · Perceptron · Practical Issues · Linear Models · Probabilistic Modeling · Neural Networks · Unsupervised Learning · Association Rules

How to use this handbook: Each topic has (a) a plain-language explanation, (b) key formulas/definitions in a box, (c) 3–5 worked examples, and (d) 5–10 practice questions in PG-entrance style. Do a full pass reading, then a second pass attempting only the practice questions.

1. Introduction to Machine Learning

1.1 What is Machine Learning?

Machine Learning (ML) is the study of algorithms that improve their performance on a task through **experience (data)**, without being explicitly programmed with fixed rules. Instead of writing "if-then" rules by hand, we give the algorithm examples and let it find the pattern.

Tom Mitchell's formal definition: A program learns from experience E with respect to task T and performance measure P , if its performance at T , measured by P , improves with E .

1.2 Types of Learning

Type	Data given	Goal	Example
Supervised	Input-output pairs (x, y)	Learn mapping $x \rightarrow y$	Spam detection
Unsupervised	Only inputs x	Find structure/patterns	Customer segmentation
Semi-supervised	Few labeled + many unlabeled	Use both to learn	Web page classification
Reinforcement	States, actions, rewards	Learn a policy maximizing reward	Game-playing agents

1.3 Key Definitions

- **Feature (attribute):** a measurable property of the data (e.g., height).
- **Label (target):** the value to predict (e.g., "spam"/"not spam").
- **Instance:** one data point (a row).
- **Training set:** data used to fit the model.
- **Test set:** data used to evaluate the model, unseen during training.

- **Hypothesis (h):** a candidate function mapping inputs to outputs.
- **Hypothesis space (H):** the set of all functions the learning algorithm can choose from.
- **Model/Concept:** the true underlying function we are trying to approximate.
- **Classification vs Regression:** discrete label output vs continuous numeric output.

1.4 Examples

1. **Email spam filter** — supervised classification; features = word frequencies; label = spam/ham.
2. **House price prediction** — supervised regression; features = area, location, age; label = price (continuous).
3. **Grouping news articles by topic** — unsupervised clustering; no labels given.
4. **Self-driving car steering agent** — reinforcement learning; reward = staying on road, avoiding collisions.
5. **Handwritten digit recognition (MNIST)** — supervised multiclass classification.

1.5 Practice Questions

1. Predicting whether a tumor is malignant or benign from cell measurements is an example of: (a) Regression (b) Classification (c) Clustering (d) Association rule mining
2. In Tom Mitchell's definition of learning, "P" stands for: (a) Probability (b) Performance measure (c) Parameters (d) Prediction
3. Grouping customers into segments without predefined labels is called: (a) Supervised learning (b) Unsupervised learning (c) Reinforcement learning (d) Active learning
4. The set of all candidate functions a learner can choose from is called the: (a) Feature space (b) Hypothesis space (c) Label space (d) Loss space
5. Which of the following is a regression task? (a) Predicting stock price tomorrow (b) Predicting email as spam (c) Detecting a face in an image (d) Clustering documents
6. A chess-playing program that improves by playing games against itself and receiving win/loss feedback uses: (a) Supervised learning (b) Unsupervised learning (c) Reinforcement learning (d) Semi-supervised learning
7. Data used only to evaluate a trained model's generalization is called the: (a) Training set (b) Validation set (c) Test set (d) Development set
8. True/False: In supervised learning, the algorithm has access to correct output labels during training.

2. Limits of Learning

2.1 Why Learning Has Limits

No learning algorithm can perfectly generalize from finite training data to *all* possible future inputs without some assumption about the world. This is the heart of the **No Free Lunch (NFL) theorem** and the **inductive bias** concept.

2.2 Key Ideas

- **Inductive Bias:** the set of assumptions a learner uses to predict outputs for unseen inputs. Without bias, learning from finite data is impossible (infinitely many functions fit any finite dataset).
- **No Free Lunch Theorem:** averaged over *all* possible problems, no single learning algorithm outperforms another; an algorithm's success depends on how well its bias matches the true problem.
- **Overfitting:** model fits training data (including noise) too closely and fails on new data.
- **Underfitting:** model is too simple to capture the underlying pattern.
- **Bias-Variance Tradeoff:** Expected Error = Bias² + Variance + Irreducible noise
 - **Bias:** error from overly simplistic assumptions.
 - **Variance:** error from sensitivity to fluctuations in training data.
- **Occam's Razor:** prefer the simplest hypothesis consistent with the data.
- **PAC Learning (Probably Approximately Correct):** a formal framework stating a concept is learnable if, with high probability ($1-\delta$), the learned hypothesis has error less than ϵ , given enough samples.
- **Sample complexity:** the number of training examples needed to learn a concept to a desired accuracy/confidence.

2.3 Examples

1. A model with high **bias** (e.g., a straight line fit to curved data) underfits — consistently wrong in the same way.
2. A model with high **variance** (e.g., a very deep decision tree) overfits — changes wildly with small changes in training data.
3. Two learners assuming different biases (linear vs. decision tree) may each excel on different types of problems — illustrating "No Free Lunch."
4. In PAC learning, if we want error $\epsilon = 0.1$ with confidence 90% ($\delta = 0.1$), the sample complexity formula tells us the minimum training examples required.
5. Adding irrelevant features increases variance without helping bias — a common overfitting cause.

2.4 Practice Questions

1. The No Free Lunch theorem implies: (a) One algorithm is always best (b) No algorithm is universally superior across all problems (c) Deep learning always wins (d) Linear models always underfit

2. High variance and low bias typically indicate: (a) Underfitting (b) Overfitting (c) Perfect fit (d) No learning occurred
 3. Inductive bias is necessary because: (a) Data is always noisy (b) Finite data is consistent with infinitely many hypotheses (c) Computers are slow (d) Labels are always wrong
 4. PAC learning framework primarily formalizes: (a) Speed of an algorithm (b) Sample complexity needed for approximately correct learning with high probability (c) Number of features needed (d) Memory usage
 5. Occam's Razor in ML suggests: (a) Always choose the most complex model (b) Prefer the simplest model consistent with data (c) Ignore training error (d) Use only neural networks
 6. Which is a sign of overfitting? (a) High training error, high test error (b) Low training error, high test error (c) Low training error, low test error (d) High training error, low test error
 7. Bias-Variance decomposition of expected squared error includes which three terms?
 8. True/False: Increasing model complexity always reduces both bias and variance.
-

3. Geometry and Nearest Neighbours

3.1 Concept

Many ML algorithms treat data points as **vectors in a geometric space**. Similar points lie close together. The **k-Nearest Neighbours (k-NN)** algorithm classifies (or predicts) a new point based on the labels of the k closest training points.

3.2 Key Formulas

Euclidean distance between points x and y in n dimensions: $d(x, y) = \sqrt{(\sum_i (x_i - y_i)^2)}$

Manhattan distance: $d(x, y) = \sum_i |x_i - y_i|$

Minkowski distance (generalized): $d(x, y) = (\sum_i |x_i - y_i|^p)^{1/p}$ ($p=1 \rightarrow$ Manhattan, $p=2 \rightarrow$ Euclidean)

k-NN classification rule: Assign the class that is the **majority vote** among the k nearest neighbours of the query point.

k-NN regression rule: Predict the **average** (or weighted average) of the target values of the k nearest neighbours.

Curse of Dimensionality: as the number of dimensions increases, data points become sparse and distances between points become less meaningful, degrading k-NN performance.

3.3 Examples

1. **k=1 NN:** Classify a new flower by the single most similar flower in the training set (nearest neighbour rule).

2. **k=5 NN with majority vote:** If 3 of 5 nearest neighbours are "cat" and 2 are "dog," predict "cat."
3. **Weighted k-NN:** Give closer neighbours higher weight (e.g., weight = $1/\text{distance}$) so nearer points influence the prediction more.
4. **Choosing k via cross-validation:** Small k (like 1) is sensitive to noise (high variance); large k oversmooths (high bias) — often the optimal k is found by trying several values and picking the one with lowest validation error.
5. **Curse of dimensionality illustration:** In 2D, "nearest" points are meaningfully close; in 1000D, almost all points end up roughly equidistant, making k-NN less reliable without dimensionality reduction.

3.4 Practice Questions

1. In k-NN, increasing k generally leads to: (a) More overfitting (b) Smoother, more biased decision boundaries (c) Faster training (d) No effect on bias/variance
 2. The Euclidean distance between (1,2) and (4,6) is: (a) 5 (b) 7 (c) 3 (d) 25
 3. k-NN is called a "lazy learner" because: (a) It has no accuracy (b) It defers computation until prediction time, storing all training data (c) It uses random guesses (d) It requires no data
 4. The curse of dimensionality refers to: (a) Too few features causing underfitting (b) Distances becoming less discriminative as dimensions increase (c) Overfitting only in 2D (d) k-NN always failing
 5. For k=1 nearest neighbour, the training error is typically: (a) Always 100% (b) Always 0% on training data itself (c) Undefined (d) Equal to test error
 6. Which distance metric sums absolute coordinate differences? (a) Euclidean (b) Manhattan (c) Cosine (d) Minkowski with $p=2$
 7. A major disadvantage of k-NN for large datasets is: (a) It cannot handle categorical data (b) High prediction-time cost due to distance computation with all training points (c) It cannot do regression (d) It requires labeled test data
 8. If k equals the total number of training points, k-NN classification always predicts: (a) The nearest point's label (b) The majority class in the entire training set (c) A random class (d) Undefined output
-

4. The Perceptron

4.1 Concept

The Perceptron is one of the earliest and simplest **linear classifiers**, mimicking a single neuron. It computes a weighted sum of inputs and passes it through a threshold (step) function to output a binary class (+1 or -1).

4.2 Key Formulas

Perceptron output: $\hat{y} = \text{sign}(w \cdot x + b) = \text{sign}(\sum_i w_i x_i + b)$

where w = weight vector, x = input vector, b = bias term.

Perceptron Learning Rule (update only on misclassification): If $\hat{y} \neq y$ (true label): $w \leftarrow w + \eta \cdot y \cdot x$ $b \leftarrow b + \eta \cdot y$ where η = learning rate.

Perceptron Convergence Theorem: If the training data is **linearly separable**, the perceptron algorithm is guaranteed to converge to a separating hyperplane in a finite number of updates.

Limitation: A single perceptron cannot solve non-linearly separable problems (classic example: the XOR problem).

4.3 Examples

1. **AND gate:** Inputs $(0,0) \rightarrow 0$, $(0,1) \rightarrow 0$, $(1,0) \rightarrow 0$, $(1,1) \rightarrow 1$ — linearly separable, perceptron learns it easily.
2. **OR gate:** Also linearly separable — a single perceptron works.
3. **XOR gate:** Not linearly separable — a single-layer perceptron *cannot* learn XOR; this historically motivated multi-layer networks.
4. **Weight update walkthrough:** For $x=(1,2)$, $y=+1$, current $w=(0,0)$, $b=0 \rightarrow$ prediction $\hat{y}=\text{sign}(0)=\text{undefined}/0$ (misclassified) $\rightarrow$ update: $w=(1,2)$, $b=1$ (with $\eta=1$).
5. **Geometric view:** The perceptron finds a hyperplane $w \cdot x + b = 0$ that separates positive and negative classes; the weight vector w is perpendicular to this hyperplane.

4.4 Practice Questions

1. The perceptron convergence theorem guarantees convergence only when: (a) Data has more than 2 features (b) Data is linearly separable (c) Learning rate is 1 (d) Data is normalized
2. Which logic gate cannot be represented by a single-layer perceptron? (a) AND (b) OR (c) XOR (d) NAND
3. In the perceptron update rule, updates occur: (a) After every example regardless of correctness (b) Only when the prediction is wrong (c) Only at the end of training (d) Never
4. The perceptron decision boundary is a: (a) Curve (b) Hyperplane (c) Circle (d) Set of disconnected points
5. The role of the bias term (b) in a perceptron is to: (a) Add noise (b) Shift the decision boundary away from the origin (c) Normalize input (d) Increase learning rate
6. If η (learning rate) is very large, the perceptron may: (a) Converge faster always (b) Oscillate and take longer/fail to converge smoothly (c) Never update weights (d) Become a regression model
7. What does " $\text{sign}(w \cdot x + b)$ " output for a perceptron? (a) A probability between 0 and 1 (b) A continuous real number (c) A binary class label (+1/-1) (d) A distance value

8. Historically, the discovery that perceptrons cannot solve XOR led to: (a) Abandoning neural networks briefly, later solved by multi-layer networks (b) Immediate invention of deep learning (c) End of ML research (d) Only regression models being used thereafter
-

5. Practical Issues in Machine Learning

5.1 Concept

Beyond the math, real-world ML requires handling messy data, choosing correct evaluation methods, and avoiding pitfalls such as overfitting, data leakage, and poor feature engineering.

5.2 Key Definitions

- **Cross-validation (k-fold):** split data into k folds; train on k-1 folds, test on remaining fold; repeat k times and average results — reduces evaluation variance.
- **Confusion Matrix:** table of TP, FP, TN, FN used to compute performance metrics.
- **Accuracy** = $(TP+TN)/(TP+TN+FP+FN)$
- **Precision** = $TP/(TP+FP)$
- **Recall (Sensitivity)** = $TP/(TP+FN)$
- **F1-score** = $2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall})$
- **Regularization:** adding a penalty term to loss to reduce overfitting (L1 = Lasso, L2 = Ridge).
- **Feature scaling:** normalizing/standardizing features (important for distance-based & gradient-based methods).
- **Data leakage:** when information from outside the training set (often the test set) improperly influences the model, giving unrealistically good performance.
- **Class imbalance:** when one class vastly outnumbers another, requiring techniques like resampling or weighted loss.
- **Hyperparameter tuning:** choosing settings (like k in k-NN, learning rate) not learned from data directly, often via validation sets or grid search.

5.3 Examples

1. **5-fold cross-validation** on 100 samples: each fold has 20 samples used once as test set.
2. **Medical diagnosis with imbalance:** 950 healthy, 50 diseased — accuracy alone is misleading; recall and F1 matter more.
3. **Data leakage example:** normalizing the entire dataset (train+test together) before splitting leaks test statistics into training.
4. **L2 regularization** added to linear regression loss: $J(w) = \text{MSE} + \lambda \cdot \sum w_i^2$ — shrinks weights, reduces overfitting.

5. **Feature scaling necessity:** in k-NN, a feature ranging 0-1000 (income) would dominate distance over a feature ranging 0-1 (normalized age) unless scaled.

5.4 Practice Questions

1. In a confusion matrix, recall is also known as: (a) Precision (b) Sensitivity/True Positive Rate (c) Specificity (d) F1 score
 2. Why is accuracy a poor metric for highly imbalanced datasets? (a) It's mathematically undefined (b) A trivial "always predict majority class" model can score high accuracy (c) It cannot be computed (d) It only works for regression
 3. K-fold cross-validation is primarily used to: (a) Increase dataset size (b) Get a more reliable estimate of model performance (c) Replace regularization (d) Speed up training only
 4. L1 regularization (Lasso) tends to produce: (a) Dense weight vectors (b) Sparse weight vectors (some weights exactly zero) (c) Negative weights only (d) No effect on weights
 5. Data leakage during preprocessing typically causes: (a) Underestimated test performance (b) Overestimated (unrealistically good) test performance (c) No effect (d) Model to fail to train
 6. F1-score is the harmonic mean of: (a) Accuracy and Recall (b) Precision and Recall (c) TP and TN (d) Bias and Variance
 7. Which technique helps address class imbalance? (a) Removing all minority samples (b) Oversampling minority class or undersampling majority class (c) Ignoring the problem (d) Increasing learning rate
 8. Standardizing (z-score normalizing) a feature transforms it to have: (a) Mean 0, standard deviation 1 (b) Mean 1, standard deviation 0 (c) Range [0,1] always (d) Only positive values
 9. Grid search is commonly used for: (a) Feature extraction (b) Hyperparameter tuning (c) Data cleaning (d) Loss function design
 10. True/False: A model with 99% training accuracy and 60% test accuracy is likely overfitting.
-

6. Linear Models

6.1 Concept

Linear models predict outputs as a **linear (weighted sum) combination of input features**. They are used both for regression (Linear Regression) and classification (Logistic Regression, Linear SVM).

6.2 Key Formulas

Linear Regression hypothesis: $\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n = \mathbf{w}^T\mathbf{x}$

Loss function (Mean Squared Error): $J(w) = (1/N) \sum_i (y_i - \hat{y}_i)^2$

Gradient Descent update: $w \leftarrow w - \eta \cdot \partial J / \partial w$

Normal Equation (closed-form solution): $w = (X^T X)^{-1} X^T y$

Logistic Regression (for classification): $P(y=1|x) = \sigma(w^T x) = 1 / (1 + e^{-(w^T x)})$ [Sigmoid function]

Logistic Loss (Cross-Entropy): $J(w) = -(1/N) \sum_i [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$

Decision rule: Predict class 1 if $P(y=1|x) \geq 0.5$, else class 0.

Support Vector Machine (SVM) — max margin idea: Maximize margin = $2/\|w\|$ subject to $y_i(w \cdot x_i + b) \geq 1$ for all i .

6.3 Examples

1. **Simple linear regression:** predicting exam score from hours studied: $\hat{y} = 5 + 10x$ (score = $5 + 10 \times$ hours).
2. **Multiple linear regression:** house price = $w_0 + w_1(\text{area}) + w_2(\text{bedrooms}) + w_3(\text{age})$.
3. **Logistic regression for spam:** $P(\text{spam}|x) = \sigma(w \cdot x)$; if σ output = 0.8, classify as spam (>0.5).
4. **Gradient descent step:** starting $w=0$, if gradient is negative, w increases in next step (moving downhill on loss surface).
5. **SVM margin maximization:** among many possible separating lines for linearly separable data, SVM picks the one maximizing distance to nearest points (support vectors) on each side.

6.4 Practice Questions

1. Linear regression minimizes which loss function by default? (a) Cross-entropy (b) Mean Squared Error (c) Hinge loss (d) KL divergence
2. The sigmoid function output range is: (a) $(-\infty, \infty)$ (b) $[0, 1]$ (c) $[-1, 1]$ (d) $\{0, 1\}$ only
3. In gradient descent, if the learning rate is too high, the algorithm may: (a) Converge slowly but surely (b) Overshoot and diverge (c) Always find global minimum (d) Behave identically to normal equation
4. Logistic regression is used for: (a) Regression only (b) Classification (c) Clustering (d) Dimensionality reduction
5. The normal equation for linear regression requires computing: (a) Gradient descent iterations (b) $(X^T X)^{-1} X^T y$ (c) Eigenvalues of covariance matrix only (d) k-NN distances
6. In SVM, "support vectors" are: (a) All training points (b) The points closest to the decision boundary that define the margin (c) Misclassified points only (d) Randomly chosen points
7. What does the cross-entropy loss penalize most heavily? (a) Correct predictions with high confidence (b) Confident wrong predictions (c) All predictions equally (d) Only ties at 0.5
8. If $w^T x = 0$ exactly for logistic regression, the predicted probability is: (a) 0 (b) 1 (c) 0.5 (d) Undefined

9. A key assumption of ordinary linear regression is: (a) Non-linearity between features and target (b) Linear relationship between features and target (c) Categorical target variable (d) No relationship needed
 10. Increasing regularization strength (λ) in Ridge regression typically: (a) Increases model complexity (b) Shrinks coefficients toward zero, reducing variance (c) Has no effect (d) Always improves training accuracy
-

7. Probabilistic Modeling

7.1 Concept

Probabilistic models treat learning as **estimating probability distributions** over data and labels, using principles from probability theory (Bayes' theorem) to make predictions under uncertainty.

7.2 Key Formulas

Bayes' Theorem: $P(A|B) = [P(B|A) \cdot P(A)] / P(B)$

Naive Bayes classifier: $P(y|x_1, \dots, x_n) \propto P(y) \cdot \prod_i P(x_i|y)$ (assumes features are conditionally independent given the class — the "naive" assumption)

Maximum Likelihood Estimation (MLE): $\hat{w} = \operatorname{argmax}_w P(\text{Data} | w)$

Maximum A Posteriori (MAP) Estimation: $\hat{w} = \operatorname{argmax}_w P(w | \text{Data}) = \operatorname{argmax}_w P(\text{Data}|w) \cdot P(w)$

Gaussian (Normal) Distribution: $P(x) = (1/\sqrt{2\pi\sigma^2}) \cdot e^{-(x-\mu)^2/2\sigma^2}$

Terminology:

- Prior: $P(y)$ — belief before seeing data.
- Likelihood: $P(x|y)$ — how probable data is given a class.
- Posterior: $P(y|x)$ — updated belief after seeing data.
- Evidence: $P(x)$ — normalizing constant.

7.3 Examples

1. **Medical test example:** disease prevalence 1%, test 99% accurate $\rightarrow$ Bayes' theorem shows $P(\text{disease}|\text{positive test})$ is much lower than 99% due to low prior — classic base-rate example.
2. **Naive Bayes spam filter:** treats each word's presence as conditionally independent given spam/not-spam, multiplies probabilities to classify an email.
3. **MLE for a coin toss:** if 7 heads out of 10 tosses observed, MLE estimate of $P(\text{heads}) = 0.7$.
4. **MAP with prior belief:** if we believe a coin is fair (prior centered at 0.5), MAP estimate pulls the MLE estimate (0.7) slightly toward 0.5 depending on prior strength.
5. **Gaussian Naive Bayes:** for continuous features (e.g., height), assume each class's feature values follow a Gaussian distribution, estimate mean & variance per class.

7.4 Practice Questions

1. Bayes' theorem relates posterior probability to: (a) Likelihood and prior (b) Only prior (c) Only likelihood (d) Sample size alone
 2. The "naive" assumption in Naive Bayes is that: (a) All features are identical (b) Features are conditionally independent given the class (c) The prior is always uniform (d) Data has no noise
 3. Maximum Likelihood Estimation chooses parameters that: (a) Minimize prior probability (b) Maximize the probability of observed data given parameters (c) Minimize the posterior (d) Ignore the data entirely
 4. MAP estimation differs from MLE by: (a) Ignoring the likelihood (b) Incorporating a prior distribution over parameters (c) Requiring infinite data (d) Being identical to MLE always
 5. In the base-rate fallacy example (rare disease, imperfect test), a positive test result usually implies: (a) Certainty of disease (b) Posterior probability of disease can still be low despite high test accuracy (c) The test is useless (d) Prior probability is irrelevant
 6. For Gaussian Naive Bayes, we typically estimate which two parameters per feature per class? (a) Mean and variance (b) Median and mode (c) Min and max (d) Skewness and kurtosis
 7. $P(x|y)$ in Bayesian terminology is called the: (a) Posterior (b) Likelihood (c) Prior (d) Evidence
 8. Naive Bayes is popular for text classification mainly because: (a) It requires huge computation (b) It is simple, fast, and works well despite the independence assumption (c) It always outperforms neural networks (d) It requires no training data
-

8. Neural Networks

8.1 Concept

Neural Networks extend the perceptron idea by stacking multiple layers of interconnected neurons, enabling them to learn **non-linear** decision boundaries and complex functions (solving problems like XOR that a single perceptron cannot).

8.2 Key Formulas

Single neuron computation: $z = \sum_i w_i x_i + b$; $a = f(z)$ where f is an **activation function**.

Common activation functions:

- Sigmoid: $f(z) = 1/(1+e^{(-z)})$
- Tanh: $f(z) = (e^z - e^{-z})/(e^z + e^{-z})$
- ReLU: $f(z) = \max(0, z)$

Feedforward computation: input $\rightarrow$ hidden layer(s) $\rightarrow$ output layer, each layer applying weights + activation.

Loss functions: MSE (regression) or Cross-Entropy (classification), same as linear models but applied to network output.

Backpropagation: algorithm to compute gradients of the loss with respect to every weight using the **chain rule**, propagating error backward from output to input layer.

Gradient descent weight update (per layer): $w \leftarrow w - \eta \cdot (\partial \text{Loss} / \partial w)$

Universal Approximation Theorem: A feedforward network with at least one hidden layer (with enough neurons and a suitable activation function) can approximate any continuous function to arbitrary accuracy.

8.3 Examples

1. **XOR solved by MLP:** a 2-layer network (1 hidden layer with 2 neurons) can represent XOR, unlike a single perceptron.
2. **Image classification:** input pixels $\rightarrow$ hidden layers extract edges/shapes $\rightarrow$ output layer gives class probabilities (e.g., "cat" vs "dog").
3. **Vanishing gradient problem:** with sigmoid/tanh in deep networks, gradients shrink across many layers, slowing learning in early layers — a key motivation for ReLU.
4. **Backpropagation walkthrough (conceptual):** compute forward pass output, compute loss, compute gradient of loss w.r.t. output layer weights, then propagate error backward layer by layer using chain rule to update all weights.
5. **Overfitting in neural nets:** a network with too many neurons/layers memorizes training data; techniques like dropout and regularization help generalize.

8.4 Practice Questions

1. The main reason multi-layer neural networks can solve XOR (unlike single perceptrons) is: (a) They use larger learning rates (b) Hidden layers with non-linear activations enable non-linear decision boundaries (c) They use more training data always (d) They avoid using weights
2. Backpropagation primarily computes: (a) Random weight initialization (b) Gradients of loss with respect to weights via the chain rule (c) The final classification directly (d) Feature scaling factors
3. ReLU activation is defined as: (a) $\max(0, z)$ (b) $1/(1+e^{-z})$ (c) $\tanh(z)$ (d) z^2
4. The vanishing gradient problem is most associated with: (a) ReLU in shallow networks (b) Sigmoid/tanh activations in deep networks (c) Linear regression (d) k-NN
5. The Universal Approximation Theorem states that a network with one hidden layer (sufficiently wide) can: (a) Only fit linear functions (b) Approximate any continuous function arbitrarily well (c) Never overfit (d) Solve only classification tasks
6. Dropout is a technique used to: (a) Speed up forward pass only (b) Reduce overfitting by randomly deactivating neurons during training (c) Increase the number of layers (d) Replace backpropagation
7. In a feedforward neural network, information flows: (a) Only backward (b) From input layer through hidden layers to output layer (c) Randomly in any direction (d) Only

within a single layer

8. Which activation function outputs values strictly between 0 and 1? (a) ReLU (b) Sigmoid (c) Linear (d) Step function
 9. Increasing the number of hidden layers (depth) generally: (a) Always improves generalization with no downside (b) Increases model capacity but risks overfitting and harder optimization (c) Has no effect on capacity (d) Reduces the need for training data
 10. The chain rule is essential in backpropagation because: (a) It computes distances between points (b) It allows gradients to be propagated through composed (nested) functions across layers (c) It normalizes inputs (d) It selects the learning rate
-

9. Unsupervised Learning

9.1 Concept

Unsupervised learning finds hidden structure in data **without labeled outputs**. The two major families are **clustering** (grouping similar points) and **dimensionality reduction** (compressing features while preserving structure).

9.2 Key Formulas & Definitions

K-Means Clustering Algorithm:

1. Choose k initial centroids.
2. Assign each point to nearest centroid: $\text{cluster}(x) = \underset{j}{\operatorname{argmin}} \|x - \mu_j\|^2$
3. Update centroids: $\mu_j = \text{mean of all points assigned to cluster } j$.
4. Repeat steps 2-3 until convergence (centroids stop changing).

K-Means Objective (minimize within-cluster variance): $J = \sum_j \sum_{(x \in C_j)} \|x - \mu_j\|^2$

Hierarchical Clustering: builds a tree (dendrogram) of clusters either bottom-up (agglomerative — merge closest clusters) or top-down (divisive — split clusters).

Linkage criteria (agglomerative): single-link (min distance), complete-link (max distance), average-link (mean distance).

Principal Component Analysis (PCA) — dimensionality reduction:

- Finds directions (principal components) of maximum variance in data.
- Computed via eigenvectors/eigenvalues of the covariance matrix, or Singular Value Decomposition (SVD).
- First principal component captures the most variance; each subsequent one is orthogonal and captures the next most.

Silhouette Score: measures how well-separated clusters are (ranges -1 to 1; higher is better).

9.3 Examples

1. **Customer segmentation:** k-means groups customers by purchasing behavior into

k=4 segments (e.g., budget, premium, occasional, frequent buyers).

2. **Image compression using PCA:** reduce a high-dimensional pixel representation to fewer principal components while retaining most visual information.
3. **Hierarchical clustering of species:** builds a dendrogram showing which species are more closely related, cuttable at any level to get k clusters.
4. **Choosing k via Elbow Method:** plot within-cluster sum of squares (J) against k; the "elbow" point (where improvement slows) suggests a good k.
5. **Document clustering:** group news articles into topics using term-frequency features and k-means, without any topic labels provided beforehand.

9.4 Practice Questions

1. K-means clustering requires which parameter to be specified in advance? (a) Number of features (b) Number of clusters (k) (c) Learning rate (d) Number of labels
 2. The k-means algorithm converges when: (a) All points belong to one cluster (b) Centroids no longer change significantly between iterations (c) k equals number of data points (d) Loss becomes exactly zero always
 3. PCA finds directions of: (a) Minimum variance (b) Maximum variance in the data (c) Random variance (d) Zero variance
 4. Agglomerative hierarchical clustering is a: (a) Top-down splitting approach (b) Bottom-up merging approach (c) Supervised technique (d) Regression technique
 5. The Elbow Method is used to: (a) Choose the learning rate (b) Determine a good value of k in k-means (c) Normalize features (d) Compute PCA components
 6. A key limitation of k-means is that it: (a) Requires labeled data (b) Assumes roughly spherical, equally-sized clusters and is sensitive to initialization (c) Cannot handle numeric data (d) Always finds the global optimum
 7. In PCA, the first principal component: (a) Captures the least variance (b) Is always equal to the mean (c) Captures the most variance in the data (d) Is orthogonal to nothing
 8. Single-linkage clustering defines cluster distance as: (a) Average of all pairwise distances (b) Minimum distance between any two points in different clusters (c) Maximum distance between any two points (d) Distance between centroids only
 9. Silhouette score close to 1 indicates: (a) Poor clustering (b) Well-separated, well-matched clusters (c) Random clustering (d) Undefined clustering
 10. True/False: Unsupervised learning requires a labeled training set to compute the loss function.
-

10. Association Rule Mining

10.1 Concept

Association Rule Mining discovers interesting **relationships (rules) between items** in large transactional datasets — famously used in market-basket analysis (e.g., "customers who buy bread also buy butter").

10.2 Key Formulas & Definitions

An association rule is written as: $X \rightarrow Y$ (if X is bought/present, Y is likely too)

Support(X): fraction of transactions containing itemset X. $\text{Support}(X) = (\text{Transactions containing } X) / (\text{Total transactions})$

Support($X \rightarrow Y$): fraction of transactions containing both X and Y. $\text{Support}(X \rightarrow Y) = (\text{Transactions containing } X \text{ and } Y) / (\text{Total transactions})$

Confidence($X \rightarrow Y$): how often Y appears in transactions that contain X.

$\text{Confidence}(X \rightarrow Y) = \text{Support}(X \cup Y) / \text{Support}(X)$

Lift($X \rightarrow Y$): measures how much more often X and Y occur together than expected if independent. $\text{Lift}(X \rightarrow Y) = \text{Confidence}(X \rightarrow Y) / \text{Support}(Y) = \text{Support}(X \cup Y) / [\text{Support}(X) \cdot \text{Support}(Y)]$

- $\text{Lift} = 1 \rightarrow X$ and Y are independent.
- $\text{Lift} > 1 \rightarrow$ positive correlation ($X \rightarrow Y$ rule is meaningful).
- $\text{Lift} < 1 \rightarrow$ negative correlation.

Apriori Algorithm principle: "All subsets of a frequent itemset must also be frequent" (anti-monotonicity/downward closure property). Used to prune the search space efficiently:

1. Find frequent 1-itemsets (support $\geq$ min_support).
2. Generate candidate k-itemsets from frequent (k-1)-itemsets.
3. Prune candidates whose subsets are not frequent.
4. Repeat until no more frequent itemsets are found.
5. Generate rules from frequent itemsets meeting minimum confidence.

10.3 Examples

1. **Market basket:** $\{\text{Bread}\} \rightarrow \{\text{Butter}\}$, Support=0.4, Confidence=0.8 means 40% of all transactions have both, and 80% of bread-buyers also buy butter.
2. **Lift calculation:** If $\text{Support}(\text{Bread})=0.5$, $\text{Support}(\text{Butter})=0.4$, $\text{Support}(\text{Bread} \cup \text{Butter})=0.3$, then $\text{Lift} = 0.3 / (0.5 \times 0.4) = 1.5 \rightarrow$ positive association.
3. **Apriori pruning example:** if $\{\text{Milk}, \text{Bread}\}$ is infrequent, then $\{\text{Milk}, \text{Bread}, \text{Eggs}\}$ cannot be frequent either — so it is pruned without checking.
4. **Recommendation systems:** "customers who bought this item also bought..." uses association rules mined from purchase histories.
5. **Web usage mining:** finding that users who visit page A frequently also visit page B, used for site navigation redesign.

10.4 Practice Questions

- Support of an itemset X measures: (a) How often Y appears given X (b) The fraction of all transactions containing X (c) The correlation coefficient (d) The number of rules generated
- Confidence($X \rightarrow Y$) is calculated as: (a) $\text{Support}(X) / \text{Support}(Y)$ (b) $\text{Support}(X \cup Y) / \text{Support}(X)$ (c) $\text{Support}(Y) / \text{Support}(X \cup Y)$ (d) $\text{Support}(X) \times \text{Support}(Y)$
- A lift value greater than 1 indicates: (a) X and Y are independent (b) X and Y are negatively correlated (c) X and Y are positively correlated (d) The rule is invalid
- The Apriori algorithm's key pruning principle is: (a) All supersets of an infrequent itemset are also infrequent (b) All subsets of a frequent itemset are also frequent (downward closure) (c) Random pruning (d) Only 2-itemsets are considered
- If $\text{Support}(\text{Bread})=0.6$ and $\text{Support}(\text{Bread} \cup \text{Jam})=0.3$, $\text{Confidence}(\text{Bread} \rightarrow \text{Jam})$ equals: (a) 0.18 (b) 0.5 (c) 0.9 (d) 2.0
- A lift value of exactly 1 means: (a) Strong positive association (b) Strong negative association (c) X and Y occur together as expected under independence (d) Rule has 100% confidence
- In market basket analysis, "minimum support" threshold is used to: (a) Filter out infrequent (rare) itemsets before rule generation (b) Compute lift directly (c) Replace confidence entirely (d) Normalize the transaction database
- Which of these algorithms is classically used for frequent itemset mining? (a) k-Means (b) Apriori (c) Perceptron (d) PCA
- True/False: A rule with high confidence always has high lift.

Quick-Revision Formula Sheet (All Topics)

Topic	Core Formula
k-NN	$d(x,y) = \sqrt{\sum (x_i - y_i)^2}$
Perceptron	$\hat{y} = \text{sign}(w \cdot x + b)$; update: $w \leftarrow w + \eta \cdot y \cdot x$
Linear Regression	$\hat{y} = w^T x$; $J = (1/N) \sum (y - \hat{y})^2$
Logistic Regression	$P(y=1$
Naive Bayes	$P(y$
Neural Net (neuron)	$a = f(\sum w_i x_i + b)$
K-Means objective	$J = \sum \sum \ x - \mu_j\ ^2$
Association Rule	Confidence = $\text{Supp}(X \cup Y) / \text{Supp}(X)$; Lift = Confidence / Supp(Y)

Final Exam-Day Tips

- Revisit the **formula sheet** above 30 minutes before the exam.
- For numerical questions (distance, support/confidence/lift, gradient steps), practice plugging numbers quickly — these are common in PG entrance MCQs.
- Watch for **conceptual traps**: overfitting vs underfitting, bias vs variance, supervised vs unsupervised — these are frequently tested via "which of the following" style questions.
- Pay special attention to **XOR + perceptron limitation** and **Naive Bayes independence assumption** — recurring favorite exam points.